



MariaDB

Additional Reading

[Visit our website](#)

What is MariaDB?

MariaDB is an open-source relational database management system (RDBMS) that serves as a robust alternative to MySQL. The original MySQL developers developed it in response to concerns about MySQL's acquisition by Oracle Corporation. MariaDB is designed to remain compatible with MySQL while offering enhanced performance, security features, and various storage engines tailored for different use cases.

Installing MariaDB

Ubuntu/Debian

1. Update the Package List:

- Before installing new software, it's crucial to ensure that your package list is up to date. Open the terminal and run:

```
sudo apt update
```

2. Install MariaDB:

- To install the MariaDB server package, execute:

```
sudo apt install mariadb-server
```

- This command will download and install MariaDB along with any required dependencies automatically.

3. Start and Enable MariaDB:

- After installation, you need to start the MariaDB service and enable it to automatically start on boot:

```
sudo systemctl start mariadb  
sudo systemctl enable mariadb
```

- The first command starts the MariaDB server, and the second ensures that it will start automatically whenever the system boots.

4. **Secure the Installation:**

- After installation, it is highly recommended to run a security script to enhance the security of your MariaDB installation:

```
sudo mysql_secure_installation
```

- This script will prompt you to:
 - Set the root password.
 - Remove anonymous users.
 - Disallow root login remotely.
 - Remove test databases.
- Follow the on-screen instructions to configure your installation securely.

5. **Verify Installation:**

- To check that MariaDB was installed correctly, you can verify the version by running:

```
mariadb --version
```

- You should see the installed version of MariaDB displayed in the terminal.

CentOS/RHEL

1. **Install MariaDB:**

- To install the MariaDB server, use the following command:

```
sudo yum install mariadb-server
```

- This command will download and install the latest version of MariaDB along with the necessary dependencies.

2. **Start and Enable MariaDB:**

- After installation, you need to start the MariaDB service and enable it to automatically start on boot:

```
sudo systemctl start mariadb  
sudo systemctl enable mariadb
```

- The first command starts the MariaDB server, and the second ensures that it will start automatically whenever the system boots.

3. **Secure the Installation:**

- Run the security script, similar to Ubuntu, to secure your installation:

```
sudo mysql_secure_installation
```

- Follow the prompts to set up a secure environment.

4. **Verify Installation:**

- Check the installation with:

```
mariadb --version
```

macOS (Using Homebrew)

1. **Install MariaDB:**

- If you have Homebrew installed, you can easily install MariaDB by executing:

```
brew install mariadb
```

- Homebrew will handle downloading and installing the necessary files.

2. **Start MariaDB:**

- To start MariaDB as a service and have it run in the background, use:

```
brew services start mariadb
```

- This command will start the MariaDB server and ensure it runs each time you start your computer.

3. **Verify Installation:**

- Confirm that the installation was successful with:

```
mariadb --version
```

- You should see the version information displayed.

Windows

1. Download MariaDB:

- Visit the official [MariaDB website](#) and download the latest installer (MSI package) that corresponds to your operating system architecture (typically x86_64 for 64-bit systems).

2. Run the Installer:

- Once the download is complete, navigate to your Downloads folder and double-click the MSI file to launch the installer. Follow these steps:
 - Accept the license agreement when prompted.
 - During the setup wizard, you will be asked to create a password for the root user. Choose a strong password and confirm it.
 - You will have the option to allow remote access for the root user; enable this if necessary for your environment.
 - Select UTF-8 as the default character set for the server to ensure compatibility with a wide range of applications.

3. Configure TCP Port:

- If you have other services (like MySQL) using the default TCP port (3306), you may need to choose a different port during the setup. For example, you can use port 3305 or 3307 if necessary to avoid conflicts.

4. Start MariaDB:

- Open Command Prompt as Administrator and run the following command:

```
net start mariadb
```

5. Secure the Installation:

- After completing the installation, use the `mysql_secure_installation` script to set up security configurations. This step will help secure your installation by removing test databases and allowing root login only from localhost.

Connecting to MariaDB

Once MariaDB is installed and running, we need to connect to it using the command-line interface or a graphical tool. Below are different ways to connect to MariaDB on various operating systems. You can choose one based on your preference. MySQL Workbench is highly recommended for a more user-friendly experience, especially for beginners.

Using the Command-Line Client

1. Open Terminal (Linux) or Command Prompt (Windows) as an administrator.
2. Ensure MariaDB is running:

For Linux:

```
sudo systemctl status mariadb
```

If the service is not running, use `sudo systemctl start mariadb` to start it.

For Windows:

```
net start MariaDB
```

3. Connect to MariaDB Server:

- To connect, use the following command:

```
mysql -u root -p
```

- The `-u` flag specifies the username (root in this case), and the `-p` flag prompts you to enter the password.

4. Enter the Root Password:

- Type the password you set during the `mysql_secure_installation` or installation process and press Enter. If the password is correct, you will be logged into the MariaDB shell.

5. Verify Connection:

- Once logged in, you can verify your connection by executing:

```
SHOW DATABASES;
```

- This command will list all databases available in your MariaDB instance, allowing you to confirm that you are connected properly.

Using phpMyAdmin

phpMyAdmin is a web-based graphical user interface(GUI) that allows you to interact with MariaDB/MySQL databases through your browser. It works on various operating systems, including Linux, Windows, and macOS.

On Windows

1. **Install XAMPP:**

- Download XAMPP from the [Apache Friends website](#) and run the installer. Ensure that you select phpMyAdmin during the installation process, as it provides a user-friendly interface for managing databases.

2. **Start Apache and MySQL (MariaDB):**

- Open the XAMPP Control Panel and start both the Apache and MySQL services. This will allow you to access phpMyAdmin via a web browser.

3. **Access phpMyAdmin:**

- Open your web browser and go to:

```
http://localhost/phpmyadmin
```

- This will open the phpMyAdmin interface, where we can manage our databases easily.

4. **Login:**

- Enter your MariaDB username (root) and the password you set during installation. Upon successful login, you'll be taken to the main dashboard.

5. **Manage Databases:**

- Once logged in, you can create new databases, manage existing ones, run SQL queries, and perform various administrative tasks using the graphical interface.

On Linux

1. **Ensure MariaDB is installed and running:**

```
sudo systemctl start mariadb
```

```
sudo systemctl enable mariadb
```

2. Install phpMyAdmin:

- Update your package list and install phpMyAdmin:

```
sudo apt update  
sudo apt install phpmyadmin -y
```

- During installation, you will be prompted to select the web server to configure. Choose Apache by pressing the space bar and then Enter.

3. Set Up phpMyAdmin:

- When prompted, choose "Yes" to configure the database and set a password for the phpMyAdmin application.

4. Access phpMyAdmin:

- Open a web browser and navigate to:

```
http://localhost/phpmyadmin
```

- Log in with your MariaDB credentials.

On macOS

1. Start MariaDB:

```
brew services start mariadb
```

2. Install PHP and Apache:

Install PHP:

```
brew install php
```

macOS includes Apache; start it with:

```
sudo apachectl start
```

3. Install phpMyadmin:

- Download phpMyAdmin from its [official site](#), unzip it, and move it to /usr/local/phpmyadmin. Configure Apache to serve it by editing /etc/apache2/httpd.conf and then add this line:

```
Alias /phpmyadmin /usr/local/phpmyadmin
```

- Restart Apache:

```
sudo apachectl restart
```

4. Access phpMyadmin:

Open a browser and go to:

```
http://localhost/phpmyadmin
```

5. Manage Databases:

Use the interface to create, edit and query databases as needed.

Using MySQL Workbench

MySQL Workbench is a popular, free, and cross-platform database management tool designed for Windows, macOS, and Linux. It provides a graphical user interface (GUI) for database administration, making it easier to manage MariaDB and MySQL databases without using the command line.

1. **Download and Install MySQL Workbench:**

- Visit the official [MySQL website](#) to download the latest version of MySQL Workbench, then follow the installation instructions provided.

2. **Create a New Connection:**

- Open MySQL Workbench and click on the + icon next to MySQL Connections to create a new connection. Ensure that MariaDB is running and enabled.

3. **Enter Connection Details:**

- Connection Name: Choose a name for your connection.
- Hostname: 127.0.0.1 (or localhost)
- Username: root

- Password: Enter your MariaDB root password.
- Port: 3306 (default MariaDB port, change if necessary).

4. **Test Connection:**

- Click on Test Connection to ensure your settings are correct. If successful, you will see a confirmation message.

5. **Connect and Manage Databases:**

- Once connected, you can use the graphical interface to manage databases, run SQL queries, and perform various administrative tasks.

Using HeidiSQL

HeidiSQL is a lightweight, free, and open-source database management tool designed for **Windows**. It allows users to connect to MariaDB, MySQL, PostgreSQL, and Microsoft SQL Server databases, making it a powerful tool for database administration and development. If you have installed MariaDB on Windows using the official installer, HeidiSQL is usually included by default. This means you may not need to install it separately.

1. **Open HeidiSQL:**

- Start the HeidiSQL application and ensure that MariaDB is running.

2. **Create a New Session:**

- Click the "New" button in the bottom-left corner to create a new session.

3. **Enter Connection Details:**

- Network Type: MySQL (TCP/IP) or MariaDB
- Hostname/IP: 127.0.0.1 (if MariaDB is running locally)
- User: root
- Password: Enter the MariaDB root password you set during installation.
- Port: 3306 (default MariaDB port, change if necessary).

4. **Connect:**

- Click the "Open" button to connect to your MariaDB instance.

5. **Manage Your MariaDB Databases:**

- Try running a simple SQL command to verify your connection:

```
SHOW DATABASES;
```

- This command will display all the databases available in your MariaDB server, allowing you to confirm your connection is working correctly.
-

Basic MariaDB Commands

Command	Description
<code>SHOW DATABASES;</code>	List all databases
<code>CREATE DATABASE mydb;</code>	Creates a database named mydb
<code>USE mydb;</code>	Switches to mydb
<code>CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100));</code>	Creates a table
<code>INSERT INTO users VALUES (1, "Alice");</code>	Adds data to the table
<code>SELECT * FROM users;</code>	Retrieves all data
<code>DROP DATABASE mydb;</code>	Deletes the database

Importing CSV files

The following instructions will guide you through importing CSV files into MySQL Workbench and HeidiSQL. While both tools are effective, MySQL Workbench is recommended as the first option due to its user-friendly interface.

MySQL

1. Open MySQL Workbench: Launch the application on your computer.
2. Create a Database: Ensure that you have created a database. If not, create one by clicking on the "Create a New Schema" button.
3. Open the Table Data Import Wizard:
 - Right-click on the database you created.
 - Select Table Data Import Wizard.

4. Select the CSV File:

- Browse to locate the CSV file you would like to import.
- Click Next.

5. Create a New Table:

- Select Create New Table within the database you created earlier.
- You can change the default name of the table if desired.
- Click Next.
- Alternatively, if you've already created a table, you can choose it from your database for the import.

6. Configure Import Settings:

- Review and configure the import settings.
- Ensure that everything appears correct, then click Next.

7. Import the Data:

- Click Next to start the import process.
- You will see the import results, indicating that the table has been created and the number of records that have been imported.

8. Finish the Process: Click Finish. You can check the table to confirm that the data has been imported successfully.

HeidiSQL

1. Open HeidiSQL: Launch the application on your computer.

Import CSV Files:

- Click on Tools in the navigation bar.
- Select Import CSV files.

2. Configure Control Characters:

- Ensure the following control characters are set:
 - Fields terminated by ,
 - Fields enclosed by: "
 - Fields escaped by: "

- Lines terminated by: \n

3. Select Destination:

- Under Destination, select the appropriate database and the table into which you want to import the data.
- Investigate the columns to ensure that everything is appropriate.
- Alternatively, you can create a new table, which is often more efficient than importing into an existing table.

4. Import the Data:

- Once you are satisfied with all the settings, click Import! Your data will now be imported into the selected table.

Importing CSV Files Using SQL

If you prefer to import CSV files using SQL commands, follow these steps:

1. Ensure the Table exists:

```
CREATE TABLE IF NOT EXISTS my_table (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  age INT,  
  city VARCHAR(100)  
);
```

The columns in the table should match the columns in the CSV file in terms of number, order, and data types to ensure a smooth import.

2. Use LOAD DATA INFILE Command:

```
LOAD DATA INFILE '/path/to/your/file.csv'  
INTO TABLE my_table  
FIELDS TERMINATED BY ','  
ENCLOSED BY '"'  
LINES TERMINATED BY '\n'  
IGNORE 1 ROWS;
```

3. Ensure File Permissions:

If the CSV file is on your **local computer**, enable local file loading in MySQL Workbench or HeidiSQL before importing:

```
SET GLOBAL local_infile = 1;
```

Saving Your Query and Results

After writing and running your SQL query in MySQL Workbench or HeidiSQL, you'll need to save both your query (the code you wrote) and the results (what the database shows) to submit your answer.

1. Copy the Query (getting your SQL code):

- MySQL Workbench: Highlight your query text in the query tab, then press Ctrl+C (or right-click > Copy) to copy it.
- HeidiSQL: In the Query tab, right-click your query text and select Copy to grab it.

2. Export or Copy Results (saving what the database returns):

- MySQL Workbench: After running your query, look at the results grid below. Right-click the grid, choose Export, and save it as a CSV file (e.g., results.csv). Or, if you prefer, right-click > Copy Row Content to copy the rows manually and paste them elsewhere.
- HeidiSQL: Run your query, then see the results in the Data tab. Right-click the grid, select Export grid rows, choose CSV, and save it (e.g., results.csv).
- If No Results Show Up: If the grid is empty (like when no songs were skipped), write a note like "No results" in your submission, or copy the empty grid structure to prove you ran it.

If you used the command line, copy and paste your SQL code into a text file and save your query results as a spreadsheet (e.g., Excel) or a CSV file.

Ensure that you create separate files for each query. Once all files are ready, upload them to your task folder and name each file according to the task it accomplishes, making it easier for the reviewer to locate your files.